Adding queues gives you finer control over the priority of jobs, but what about processing more jobs at once?

Use Concurrency to Process More Jobs at Once

In the example above, two orders were created at roughly the same time, but because we were only processing one job at a time, the second order had to wait. If you recall from *Set Sidekiq's Configuration Options*, we configured a value for `concurrency:` with a default of 5. This value controls the number of threads each Sidekiq process will use.

Threads are bits of code running concurrently, but inside the same Ruby process. Threads share memory, processes do not. The way we run the example app locally runs one Sidekiq process that spawns five threads. This means that our default configuration can process five jobs at once (all of which share the same memory—this will be relevant in a moment).

Suppose `CompleteOrderJob` takes two seconds to complete. If we got six orders at once, five of the users would wait two seconds, and one would have to wait four (two for at least one other job to finish and two for their job). If we got eleven orders at once, the eleventh order would have to wait six seconds.

In theory, you can adjust the number of processes and/or the number of threads per process to meet any demand. In practice, you have to do this carefully since each thread and process consumes system resources like CPU or memory. Tuning these two aspects of concurrency can be difficult and time-consuming. It highly depends on what your app does and how it is hosted. But you can make that job a bit easier by being mindful of how concurrency has the potential to create serious problems, even at small scale.

An Example Concurrency Problem

Consider the following code, which stores a value in a class variable.

```
class Fulfiller
  def start_fulfillment(order)
    self.num_fulfilled = self.num_fulfilled + 1
  end
```